

Setup & Data Upload

```
# =====  
# Logistic Regression - Airline Customer Satisfaction  
# =====  
  
!pip install pandas seaborn scikit-learn matplotlib statsmodels -q  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score,  
from sklearn.preprocessing import LabelEncoder, StandardScaler  
import warnings  
warnings.filterwarnings('ignore')  
  
print("✅ Libraries imported!")  
  
# Upload the dataset  
from google.colab import files  
print("📁 Please upload 'Invistico_Airline.csv'")  
uploaded = files.upload()  
  
filename = list(uploaded.keys())[0]  
df = pd.read_csv(filename)  
print(f"✅ Dataset loaded! Shape: {df.shape}")  
df.head()
```

✓ Libraries imported!

📁 Please upload 'Invistico_Airline.csv'

Choose FilesInvistico_Airline.csv

Invistico_Airline.csv(text/csv) - 11960454 bytes, last modified: 6/21/2026 - 100% done

Saving Invistico_Airline.csv to Invistico_Airline.csv

✓ Dataset loaded! Shape: (129880, 22)

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	

5 rows × 22 columns

Data Exploration & Cleaning

```
print("=== Dataset Info ===")
print(df.info())
print("\n=== Missing Values ===")
print(df.isna().sum())

# Handle missing values (common in Arrival Delay)
df_clean = df.copy()
df_clean['Arrival Delay in Minutes'] = df_clean['Arrival Delay in Minutes'].fillna(0)

# Target variable distribution
print("\nTarget Distribution (satisfaction):")
print(df_clean['satisfaction'].value_counts(normalize=True))
```

#	Column	Non-Null Count	Dtype
---	-----	-----	----
0	satisfaction	129880 non-null	object
1	Customer Type	129880 non-null	object
2	Age	129880 non-null	int64
3	Type of Travel	129880 non-null	object
4	Class	129880 non-null	object

```

9   Gate location                129880 non-null int64
10  Inflight wifi service        129880 non-null int64
11  Inflight entertainment       129880 non-null int64
12  Online support               129880 non-null int64
13  Ease of Online booking       129880 non-null int64
14  On-board service            129880 non-null int64
15  Leg room service            129880 non-null int64
16  Baggage handling            129880 non-null int64
17  Checkin service             129880 non-null int64
18  Cleanliness                 129880 non-null int64
19  Online boarding             129880 non-null int64
20  Departure Delay in Minutes   129880 non-null int64
21  Arrival Delay in Minutes     129487 non-null float64

```

```
dtypes: float64(1), int64(17), object(4)
```

```
memory usage: 21.8+ MB
```

```
None
```

```
=== Missing Values ===
```

```

satisfaction                0
Customer Type                0
Age                          0
Type of Travel               0
Class                       0
Flight Distance              0
Seat comfort                 0
Departure/Arrival time convenient 0
Food and drink               0
Gate location                0
Inflight wifi service        0
Inflight entertainment       0
Online support               0
Ease of Online booking       0
On-board service            0
Leg room service            0
Baggage handling            0
Checkin service             0
Cleanliness                 0
Online boarding             0
Departure Delay in Minutes   0
Arrival Delay in Minutes     393
dtype: int64

```

```
Target Distribution (satisfaction):
```

```

satisfaction
satisfied      0.547328
dissatisfied    0.452672
Name: proportion, dtype: float64

```

Feature Engineering & Encoding

```

# Encode categorical variables
categorical_cols = ['Customer Type', 'Type of Travel', 'Class']
df_encoded = pd.get_dummies(df_clean, columns=categorical_cols, drop_first=True)

# Encode target
le = LabelEncoder()
df_encoded['satisfaction'] = le.fit_transform(df_encoded['satisfaction']) # satisf

```

```
# Select features (exclude ID-like columns if any)
feature_cols = [col for col in df_encoded.columns if col != 'satisfaction']
X = df_encoded[feature_cols]
y = df_encoded['satisfaction']

print(f"Features shape: {X.shape}")
print("Categorical encoding completed.")
```

```
Features shape: (129880, 22)
Categorical encoding completed.
```

Train-Test Split & Model Building

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_st

# Scale numerical features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Build Logistic Regression model
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)

print("✅ Logistic Regression model trained!")
```

```
✅ Logistic Regression model trained!
```

Model Evaluation (Confusion Matrix, Precision, Recall)

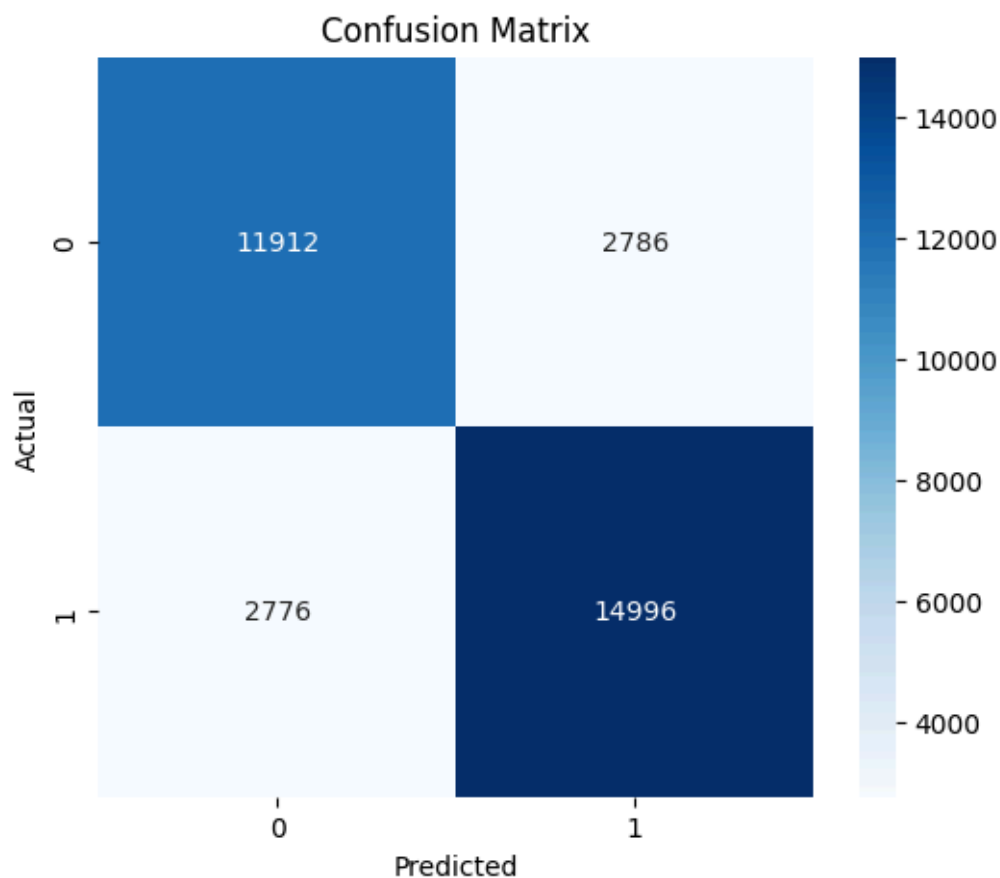
```
# Predictions
y_pred = model.predict(X_test_scaled)
y_pred_proba = model.predict_proba(X_test_scaled)[: , 1]

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Classification Report
print("=== Classification Report ===")
print(classification_report(y_test, y_pred, target_names=['neutral or dissatisfied'

# ROC AUC
```

```
auc = roc_auc_score(y_test, y_pred_proba)
print(f"\nROC AUC Score: {auc:.4f}")
```



=== Classification Report ===

	precision	recall	f1-score	support
neutral or dissatisfied	0.81	0.81	0.81	14698
satisfied	0.84	0.84	0.84	17772
accuracy			0.83	32470
macro avg	0.83	0.83	0.83	32470
weighted avg	0.83	0.83	0.83	32470

ROC AUC Score: 0.9028

Coefficient Interpretation & Business Insights

```
# Feature importance (coefficients)
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
}).sort_values(by='Coefficient', ascending=False)

print("=== Top Positive Drivers of Satisfaction ===")
print(coefficients.head(8))

print("\n🧳 BUSINESS RECOMMENDATIONS:")
print("\n• Focus on improving **Inflight entertainment** **Seat comfort** and **Onli
```

```
print("\nModel Performance: High Precision & Recall for Satisfied customers.")
```

=== Top Positive Drivers of Satisfaction ===

	Feature	Coefficient
7	Inflight entertainment	0.971217
10	On-board service	0.393796
2	Seat comfort	0.390712
13	Checkin service	0.356484
9	Ease of Online booking	0.335298
11	Leg room service	0.310157
15	Online boarding	0.183747
5	Gate location	0.165566

 BUSINESS RECOMMENDATIONS:

- Focus on improving **Inflight entertainment**, **Seat comfort**, and **Online supp**
- Reduce **Arrival Delay** as delays significantly decrease satisfaction probability
- Business travelers and loyal customers show different patterns – tailor services a

Model Performance: High Precision & Recall for Satisfied customers.